Domestic rich canonical S3 replay

This offline verifier compares independently executed legacy HTTP outputs with native weather responses rebuilt from canonical S3 raw records. The rich corpus adds station/AWS/city/ASOS, AirKorea observations/forecast/GIF, life/KASI/MFDS and stored Google air fallback to the core grid/RSS/middle-range inputs.

It supports smoke, paired replay and disposable disk-cache recovery. It does not prove production startup, AWS/EC2 cold p95, quota headroom, seven-day live shadow or cutover readiness. S20 must verify recovery to an earlier server2 release/config; rollback here is UNSUPPORTED and fails closed without introducing legacy fallback.

Run the verifier

From the repository root, use the pinned Rust toolchain and Python 3.11:

```
cargo build --manifest-path server2/Cargo.toml --locked --release
cargo build --manifest-path server2/Cargo.toml --locked --release \
  --example kma_s3_replay
python3.11 server2/tools/shadow/replay.py \
  --binary server2/target/release/server2 --family kma-rich --self-check \
  --report reports/kma-rich/replay.json
```

The selected binary anchors its build directory. The verifier executes the adjacent `examples/kma_s3_replay` library HTTP example and checks its hash before and after the run; it does not start the production main executable. CI uses release artifacts. Selected local evidence uses an explicitly labeled debug example. A stable example is required; do not build into it during verification.

No `.env`, real provider key, DB, AWS or deployment is used. Test-only transports publish public fixture raw through normal production validators and conditional CatalogStore writes into an owned loopback S3 peer. Provider HTTP is disabled for weather recovery. No production precollection, verification mode or endpoint setting is added.

Requests and recovery

The frozen Node 16.20.2 recorder executes actual legacy collectors, Mongoose wire writes/readers, direct HTTP routes and the repository gateway/geocoder. The native side receives the same public raw inputs. Rust output never supplies expectations.

23 unique requests include direct v000903/v000901/v000803/v000705, Korean/English, units, both air forecast selections, Seoul/Incheon/Jeju, HEAD/conditional GET and two actual v000903 gateway coordinate requests. Four phases yield 92 responses, not 92 independent inputs. Core55×4 and the original98 assembly-only cases remain separate. ASOS deliberately retains rejected/missing raw rows; no full historical coverage is implied.

Phase / Disk before fresh RAM/source / Expected result
cold-empty-ram-and-disk / Empty / Restore weather from canonical S3
warm-disk-empty-ram / Verified raw copies / Same response; disk hits observed
corrupt-disk-empty-ram / Truncated copies / Same response; detect and replace corruption
lost-disk-empty-ram / Deleted copies / Same response; restore from S3
No source or exact-coordinate geocoder is warmed before a request. All 84 direct responses require zero weather-provider/legacy/external HTTP. Eight coordinate responses additionally resolve the public request with exactly one owned-loopback Kakao HTTP call each, through the actual native factory/parser/quota path. Exact request coordinates remain in ephemeral memory. The frozen baseline has no replay-time geocoder calls; this explicit acquisition difference is checked against the manifest. Unexpected extra/missing calls fail the run.

PASS requires exact status, body bytes and functional-header union, all expected records exactly once, actual disk hits/corruption, zero weather-provider/external/ legacy HTTP and detection of a deliberately perturbed expected body. Only Date, Connection, Set-Cookie and frozen-server headers are declared transport differences. Content-Length and ETag remain exact, including HEAD and 304.

The same capability is available through smoke:

```
python3.11 server2/tools/shadow/smoke.py \
  --binary server2/target/release/server2 --family kma-rich \
  --report reports/kma-rich/smoke.json
```

Failure behavior and limits

The rich inputs exposed two genuine restore defects: first-fresh S3 KASI data was not read before the snapshot, and same-hour AWS/city records did not follow latest acquisition plus Mongoose `$set` presence semantics. Focused Red/Green tests cover both. Cold requests also exposed unnecessary sequential waits: KASI area reads now run in bounded groups of eight and apply in AREAS order, while image recovery starts after its own station/announcement dependencies without waiting for observations. The one combined request budget and 200 ms reserve stay unchanged.

Missing/late inputs retain the partial legacy missing-data response. No deadline or expected response was relaxed to obtain a match. The earlier 88/92 run with four cold partial responses remains recorded; its later corrected result is a separate candidate measurement. Debug loopback timing and ~149 KB synthetic GIFs do not establish EC2/release performance for real 5-6 MB images. No decoded cache or pack was added; O01 remains measurement-driven.

See fixture provenance (../../server2/tests/domestic/fixtures/S3-RICH-PROVENANCE.md), selected evidence (../evidence/tasks/issue-2742-s15b4b3-rich/verification.md) and the sequence diagram (../architecture/diagrams/server2-kma-rich-replay.html).

Actual rich replay result capture

```
S15b-4b3 RICH | ACTUAL OWNED-LOOPBACK DEBUG HTTP
UTC 2026-10-16T12:31:50+00:00
Three regions; station/ASOS, air, life/KASI/MFDS, GIF + coord
23 unique cases x 4 phases = 92 responses
Exact status / body bytes / functional header union: PASS
Unexplained differences: 0
Legacy HTTP: 0 | Weather provider HTTP: 0
Owned geocoder HTTP: 8 | External HTTP: 0
Deliberate expected-body perturbation detected: 8 differences

cold-empty-ram-and-disk: p95 1885.76 ms (23)
warm-disk-empty-ram: p95 1884.17 ms (23)
corrupt-disk-empty-ram: p95 1863.69 ms (23)
lost-disk-empty-ram: p95 2046.26 ms (23)

Debug example using production Gateway router / injected dependencies
Not AWS/EC2 latency, production startup, live shadow or rollback evidence
98 assembly-only/core55 cases separate; weather S3 restore; cold geocoder counted
```